PROVIYAA · VINII PARTNER · ARCHITECTURE AUDIT

How the mobile POS handles orders today

A forensic, read-only map of how VINII Partner creates and processes restaurant transactions, and a complete register of every database migration that is not applied.

Application	VINII Partner (Flutter), folder `vinii_partner/`
Code audited at	commit `da0a823` (2 Oct 2026)
Database evidence	Migration files on every branch of Repo-1-, One Local (OL-ui) and proviyaa-pos; production column snapshot of 22 Sep 2026
Method	Source and migrations read line by line. No code, migration or data was changed. The live database was not queried.
Report date	2 October 2026

0 of 6
database functions the phone calls (order, payment, status, bill, table code, publish) are applied

7
migration files written but not applied

1
working transaction path today: One Local delivery orders

0
inventory writes, offline queue, live subscriptions, printer or scanner integrations

Evidence labels used throughout: **[code]** read in this app's source · **[ledger]** read in a migration file · **[snapshot]** matches the 22 Sep production column list · **[unverified]** not checked. File and line references are relative to `vinii_partner/lib/` unless stated.

Contents

OVERVIEW

In plain words

14 — Application summary

15 — Migrations not applied (complete register)

DETAILED AUDIT

01 — Database

02 — Order flow

03 — KOT (kitchen order ticket) flow

04 — Payment flow

05 — Inventory flow

06 — Offline and sync

07 — API and RPC map

08 — Realtime

09 — Hardware

10 — Row security and permission enforcement

11 — Entity ownership

12 — Duplicates and conflicts

13 — Unknown or unverified

In plain words

> **Bottom line.** The phone app looks finished, but it cannot yet take a single restaurant order, payment or bill change of its own. Every one of those buttons calls a database function that has not been applied, so staff see "Not available yet". The only orders that really move through the phone today are One Local delivery orders, which One Local's own server processes.

- **What works:** sign-in, choosing the restaurant, the menu (add, edit, sold-out switch, photos), bill and GST settings, switching Swiggy/Zomato channels on or off, the team list, and the full One Local delivery cycle: ring, accept or reject, ready, out for delivery, delivered.
- **What waits for the database:** sending an order to the kitchen, holding an order, taking cash/UPI/card or split payments, removing a service charge on a saved bill, issuing table QR codes, publishing the storefront, coupons, "tables around you", and online payments.
- **Not in the app at all:** stock or inventory, working offline, live updates (screens check every few seconds instead), receipt or kitchen printers, barcode scanners, cash drawers and card terminals.
- **Things to fix before real restaurants:** a delivered One Local order stays "open" on the counter all day; the delivery fee is counted inside the restaurant's cash; the database lets any staff member change prices, orders and payments directly; and any signed-in person can replace another restaurant's menu photos.
- **Migrations:** seven migration files are written but not applied (one storefront file in the OS repository, five One Local files and one Windows POS file that must not be applied as written). The five order, payment and table functions the phone calls, and a sixth that One Local's server needs, have no migration written at all; their exact shapes are documented for the founder. See section 15.

14 — Application summary

Application. VINII Partner (`vinii_partner/`), a Flutter app for restaurant staff: phone POS, the One Local delivery inbox, menu, storefront studio, bill settings, team and table QR codes.

Audited at: commit `da0a823`, 2026-10-02. Read-only. No code, migration or data was changed.

Evidence:

- this app's source;
- migration files in `Repo-1-/supabase/migrations` (OS and POS tables) and `OL-ui/db/migrations` (One Local);
- One Local's partner API source;
- the production column snapshot of 2026-09-22.

Shape in one paragraph. The app is an **online-only client** of the shared Supabase database. It has no local database, outbox, realtime subscription, edge function or hardware driver.

- It **writes directly** only to configuration-type tables: menu, bill settings, channel switches, team roles and status. It also uploads photos to storage.
- **Every transactional write** (create order, take payment, change status, adjust bill, issue a table code, publish the storefront) goes to a database function **that no applied migration defines**. The five `pos_*` functions are not written anywhere; the storefront publish function is in an unapplied draft. Those actions currently end in "Not available yet".
- **The only working transaction path is One Local delivery orders.** The phone calls One Local's partner API, whose server runs the existing `ol_partner_act` with the service role.

Answers

What creates an order?

- **Today:** only One Local's `ol_create_restaurant_order`, run by One Local's server. For this app that means Settings → "Send test order" (`POST /api/partner/test-order`). Real One Local checkouts happen outside this app.
- **Not working:** the phone POS ("Send to kitchen" or "Save & hold") and a guest on a handed-over phone both call `pos_place_order`, which does not exist. No order is created.

What changes an order?

- One Local's `ol_partner_act`, reached from the phone's Accept, Reject, Ready, Out and Delivered buttons.
- `ol_expire_unanswered`, which runs on every inbox poll, cancels late orders.
- On the phone POS, `pos_adjust_bill` and `pos_record_payment` would change orders, but both are missing. `pos_set_order_status` is defined in the code but no screen calls it.
- The database itself would also let any member update `orders` directly (10 §3). This app never does.

What creates a KOT?

- `ol_partner_act` on **accept** of a One Local order: `next_org_seq(org,'kot')`, an insert into `kot_tickets`, and `order_items.kot_ticket_id` set.
- The phone POS's `fire_kot: true` would ask `pos_place_order`, which is missing.
- This app never writes `kot_tickets` itself.

What changes a KOT? `ol_partner_act`: **ready** sets `ready`; **out** sets `delivered`. Nothing in this app or in One Local sets `preparing`. No KOT can be cancelled or reprinted.

What marks food ready? "Mark ready" on a One Local order (`delivery_order_screen.dart:207`) → `ol_partner_act(ready)`. It sets `kot_tickets` and `order_items` to `ready`, `orders` to `ready`, and `ol_orders` to `packed`. Phone POS orders have no ready step.

What creates a payment?

- One Local "Delivered" → `ol_partner_act(delivered)` inserts one `payments` row: cash, `amount = orders.total`, `reference = 'onelocal:<id>'`.
- The phone POS's "Charge" (single or split) calls `pos_record_payment`, which is missing.
- Guest online payment (Cashfree) exists in code but is not connected.

What completes a payment?

- **One Local:** the same `delivered` step. It sets `orders.payment_method = 'cash'`, `paid_at`, status `delivered`, and `ol_orders.payment_status = 'paid'`.
- **Phone POS:** the missing `pos_record_payment` would be expected to set `status = paid`. The phone counts an order as paid when `status = 'paid'` or the payments cover the total.

When is inventory changed? Never. No inventory or stock table exists in the app, either ledger or the production snapshot. The only related writes are on/off availability flags (`menu_items.is_available`).

What happens offline?

- Nothing is saved. Sending an order, taking a payment, or any other write fails with an "offline" message.
- The in-memory cart and the saved idempotency key are kept, so a later tap retries the same request.
- The inbox keeps its last list and keeps polling. Sign-out still works.

How does sync work? There is no sync. Every action is a live request. "Live" screens poll:

- the delivery inbox every 5 s while accepting, every 30 s while not;
- guest order tracking every 5 s.

No realtime channel is used, even though `orders`, `kot_tickets`, `order_items`, `order_events`, `service_requests` and `table_sessions` are in the realtime publication.

What IDs are generated locally?

- Idempotency keys (UUID v4):
  - `order:<orgId>` and `payment:<orderId>` (plus `:2` for a split), saved in SharedPreferences;
  - the guest order key, in memory only;
  - a new `idempotency-key` on each partner API write.
- An `x-request-id` (UUID v4) for every HTTP request.
- Storage file names: `<orgId>/<itemId>-<epoch ms>.<ext>`.
- The Studio preview's stand-in order id `preview-<key>`, never stored.
- **No entity id** (order, line, KOT or payment) is generated on the phone.

What IDs come from Supabase? All entity ids, through `gen_random_uuid()` defaults: organizations, memberships, menu, tables, orders, lines, KOTs, payments and events. Also:

- order numbers (`order_no`) and KOT numbers (`kot_no`), from `org_counters` through `next_org_seq`;
- `display_no`, made by the creating function (One Local uses `OL-00024`);
- the user id, from Supabase Auth;
- the version tokens (`orders.updated_at`, `org_bill_settings.updated_at`).

What tables are written?

- **Directly by the app:**
  - `menu_categories` (insert, update, delete)
  - `menu_items` (insert, update)
  - `org_bill_settings` (upsert, update)
  - `channel_integrations` (update)
  - `org_users` (update role and status)
  - `vinii_storefronts` (upsert and update; the table is only in an unapplied draft)
  - the storage bucket `item-images`
- **Indirectly, through One Local's API:**
  - `orders`, `order_items`, `kot_tickets`, `order_events`, `payments`
  - `ol_orders`, `ol_merchant_channels`, `org_counters`
  - `delivery_jobs` and the related dispatch tables (applied status unverified)

What tables are read?

- **Directly:**
  - `org_users`, `roles`, `organizations`, `org_modules`, `role_module_access`
  - `menu_categories`, `menu_items`

    - `dining_tables`, `dining_areas`
    - `orders`, `order_items`, `payments`
    - `org_bill_settings`, `channel_integrations`
    - `vinii_storefronts` and `vinii_storefront_versions` (both only in an unapplied draft)
- **Through One Local's API:** `ol_orders`, `orders`, `order_items`, `order_events`, `ol_merchant_channels`, `org_users`.

What events are emitted?

- **The app emits none itself.** It writes no `order_events` rows and publishes nothing.
- One Local's functions, triggered by the phone, write these `order_events` kinds: `placed` (test order), `accepted`, `kot_fired`, `rejected`, `kot_ready`, `out_for_delivery`, `delivered`, `paid` and `timed_out`.
- The missing `pos_*` functions would be expected to emit others; not verifiable.

What entities appear duplicated?

- **Orders:** One Local orders show in both the Orders tab and the Sell tab's "Open orders". Delivered One Local orders stay "open" all day there, because their final status is `delivered`, not `paid`.
- **Pricing rules:** One Local uses a fixed 5% GST and adds the delivery fee into the total; the phone POS uses `org_bill_settings`.
- **Availability:** two flags, `menu_items.is_available` and `item_channel_status`.
- **Status:** the same kitchen state is held in four places: `kot_tickets`, `order_items`, `orders` and `ol_orders`.
- **Customers:** free text on each order, with no shared record.
- **Organization choice:** two rules, `current_org_id()` with `limit 1` versus the partner API's earliest membership.
- **Payments:** no idempotency column, and a non-unique `reference`.
- **Storefront draft:** in the database (if present) and on the phone.

Details are in `12_DUPLICATES.md`.

15 — Migrations not applied (complete register)

Scope. Every database change that VINII Partner, or a service it talks to, depends on, and that is not applied to the shared database.

How this was found

- Every `.sql` file on **every branch** of `Repo-1-`, `OL-ui` and `proviyaa-pos` was listed and read.
- Applied status comes from each file's header and from the repositories' own journals.
- **Production was not queried.** "Applied" and "not applied" are as recorded, not checked live.
- Per the founder's standing instruction, **no SQL was written**. Section C lists names and shapes only.

A. Applied (for reference)

Migrations	Where	Status	Evidence
`0001 – 0020`	Repo-1- `supabase/migrations` (the single ledger, foundations D7)	**Applied**	Listed in production `schema_migrations` (snapshot 2026-09-22)
`0021_one_local_core`, `0022_partner_orders`, `0023_partner_order_functions`, `0024_order_items_audit_refunds`	OL-ui `db/migrations` (branch `feat/expo-router-api`)	**Applied, but not recorded**	One Local journal: "0021–0024 are confirmed live in production". Not in `schema_migrations` and not copied into the Repo-1- ledger (foundations D7)

B. Migration files that are written but NOT applied

#	File	Repo · branch	What it creates	Needed by	Status evidence
B1	`0021_vinii_storefronts.sql`	Repo-1- · `feat/experience-cms` (worktree `Repo-1--experience-cms`)	Tables `vinii_storefronts`, `vinii_storefront_versions` (an append-only trigger blocks updates). Functions `vinii_is_member`, `vinii_can_edit_storefront`, `vinii_can_publish_storefront`, `vinii_publish_storefront(p)`. A private bucket `storefront-media` (5 MB). Row security by role	**VINII Partner** Studio (save draft, publish), Master storefront CMS, guest QR page	Header: "STATUS: DRAFT, NOT APPLIED. Needs the founder's approval … and a final number"
B2	`0025_ol_payments.sql`	OL-ui · `feat/expo-router-api`	Table `ol_payment_events`; Razorpay columns on `ol_orders`	One Local online payments	One Local journal 2026-09-27: "written but not applied"
B3	`0026_ol_checkouts.sql`	OL-ui · `feat/expo-router-api`	Table `ol_checkouts`; function `ol_checkout_payment_to_orders`; columns on `ol_orders`, `ol_coupons`, `ol_payment_events`	One Local multi-store checkout	Header: "DRAFT … NOT APPLIED". **Needs B2 first**
B4	`0027_ol_privacy.sql`	OL-ui · `feat/expo-router-api`	Tables `ol_consents` (append-only, via function `ol_consents_append_only`) and `ol_privacy_requests`	One Local DPDP consent and requests	Header: "DRAFT … NOT APPLIED"
B5	`0028_ol_marketplace_listings.sql`	OL-ui · `feat/expo-router-api`	On `ol_stores`: `status` (draft, pending, live, paused), `location_id`, `cover_image`, `lat`, `lng`, `delivery_radius_km`, `approved_by`, `approved_at`, `updated_at` (+ trigger); a unique live-listing index; a city/status index	Master "One Latur" listings (which answer "Apply migration 0028 first" until then)	Header: "DRAFT … NOT APPLIED to any database"
B6	`0029_ol_dispatch.sql`	OL-ui · `feat/expo-router-api`	Tables `riders`, `rider_store_links`, `rider_availability`, `push_tokens`, `org_dispatch_settings`, `delivery_jobs`, `delivery_assignments`, `delivery_events`. Functions `dispatch_km`, `dispatch_create_job`, `dispatch_release`, `dispatch_offer_next`, `dispatch_answer`, `dispatch_cancel`, `dispatch_due_jobs`	Rider dispatch. One Local's accept and reject hooks call it, but when it is missing the hook skips quietly (`dispatch.ts:346–352`), so the phone is unaffected	Header: "DRAFT: applied nowhere". **Needs B2 and B3 first**
B7	`20260918_create_online_orders.sql`	proviyaa-pos · `main`	Table `online_orders` with **anonymous full read and write** (`using (true)`)	Windows POS online-orders screen	Not in production (checked by the One Local session, 2026-09-28). One Local journal: "must not be applied as written"

Numbering clash.

- Repo-1-'s draft B1 is numbered `0021`.
- One Local's applied `0021_one_local_core` and its drafts `0022–0029` use the same range.
- Under foundations D7 the Repo-1- ledger assigns the real numbers; every OL-ui number is a placeholder.

Other SQL found, not part of this platform's ledger

- `OL-ui` branch `main`: `backend/schema.sql`. An older One Local schema with its own `users` table, from before the shared-database plan. Superseded.
- `Repo-1-` remote branch `backup/remote-before-restore`: `saas_mobile_app/supabase/migrations/2026051x…` and `20260630_meeting_room_booking_comment.sql`. These belong to a different application (meeting-room booking).

C. Changes the phone app needs that have NO migration written (founder to write)

The exact call shapes are in `docs/PLATFORM_CONTRACT_ALIGNMENT.md` (§4.1, §4.5), `docs/GUEST_SERVER_CONTRACT.md`, `docs/PAYMENTS_CASHFREE.md` and `docs/RIDER_DISPATCH_CONTRACT.md` (the last is not yet committed by the session that wrote it). Each function takes one `jsonb` argument `p` and raises `POS_<CODE>: message`.

C1. Order functions (POS-owned)

Function	`p` (keys)	Phone today (what staff see)
`pos_place_order`	`organization_id`, `idempotency_key`, `service_type`, `table_id?`, `guest_count?`, `customer_name?`, `customer_phone?`, `notes?`, `scheduled_for?`, `placed_by`, `fire_kot`, `lines[{menu_item_id, qty, notes?, variant_id?}]`, `waive_service_charge?`, `waive_reason?`, `coupon_code?`. **Service role only:** `sales_channel`, `display_prefix`, `external_ref`, `actor_user_id`, `request_id`, `app`	"Send to kitchen" / "Save & hold" → *Not available yet*
`pos_record_payment`	`order_id`, `method`, `amount`, `idempotency_key`, `reference?`, `expected_updated_at` (`method 'online'` service role only)	"Charge" (single and split) → *Not available yet*
`pos_set_order_status`	`order_id`, `to`, `expected_updated_at`, `reason?`	No screen calls it yet
`pos_adjust_bill`	`order_id`, `expected_updated_at`, `waive_service_charge`, `waive_reason?`	"Remove service charge" on a saved order → *Not available yet*
`pos_issue_table_qr`	`table_id`, `replace?`	"Issue code" / "Replace code" → *Not available yet*
`pos_mark_handed_over`	`order_id`, `expected_updated_at`, `request_id?`, `actor_user_id?`	Not called by the phone. Needed by One Local's server for "out for delivery"

C2. Columns and value lists

Change	Why
`orders.idempotency_key`, unique per organization when present	A retried order must not create a second order (gap G4)
`payments.idempotency_key`, unique per organization when present	A retried payment must not be recorded twice
`orders.service_charge_waived` (boolean, default false) and the waived amount	The phone reads `service_charge_waived` and `service_charge_waived_amount` (`live_pos.dart:339–340`)
Which coupon was used, on the order	Guest coupon codes (C4)
Add `online` to `payments.method` and to `orders.payment_method`	Gateway payments (C5)

C3. Access-rule fixes found by the audit (10_RLS_SECURITY.md)

Change	Gap
`is_org_member(org)`, used in place of `current_org_id()` in member policies	G1: a person in two restaurants sees only one, chosen arbitrarily
`has_org_role(org, roles[])`, with role-aware write policies on `menu_*`, `orders`, `order_items`, `kot_tickets`, `payments`, `org_bill_settings`, `channel_integrations`	G2: any member can change prices, orders and payments directly
`next_org_seq` checks that the caller belongs to `org`	Any signed-in user can advance another restaurant's counters
`item-images` storage policies limited to the caller's `<organization_id>/` folder	Any signed-in user can replace or delete any restaurant's photos
A status-move guard on `orders` (after the POS owner confirms the allowed moves)	Nothing stops a status moving backwards
`org_owner` allowed where `org_admin` is (decision Q3)	G5: owners cannot update their organization or team

C4. Guest features (`GUEST_SERVER_CONTRACT.md`)

- **Functions:**
  - `guest_nearby_orders(p {organization_id, table_id, since?, limit})`
  - `guest_similar_table_orders(p {organization_id, table_id, party_size, limit})`
  - `guest_validate_coupon(p {organization_id, code, service_type, lines, table_id?})`
- **Tables:**
  - `org_guest_settings` (`nearby_orders_enabled`, `similar_tables_enabled`, `coupons_enabled`)
  - `coupons`
  - `coupon_redemptions`
- **Phone today:** none of the three features is shown.

C5. Online payments (`PAYMENTS_CASHFREE.md`)

- **Functions:** `pos_start_online_payment(p)` and `pos_online_payment_status(p {attempt_id})`.
- **Table:** `online_payments` (`attempt_id` unique, `provider_order_id` unique, `order_ids uuid[]`, `amount`, `status`, `payer_phone`, …). Only the service role writes it.
- **Server:** a signed webhook endpoint (server code, not a migration).
- **Phone today:** no Pay button; the bill tells guests to pay staff.

C6. Rider dispatch, beyond draft B6 (`RIDER_DISPATCH_CONTRACT.md`)

- **Tables:** `delivery_proofs`, `delivery_exceptions`, `rider_cash_ledger`, `rider_earnings`.
- **Columns:** `delivery_jobs.idempotency_key` (unique), `delivery_events.v`.
- **Server endpoints:** `/api/dispatch/jobs`, `release`, `cancel`, `GET`.
- **Phone today:** unaffected.

D. Dependencies between them

- B3 needs B2. B6 needs B2 and B3.
- C1 needs the two key columns in C2 to keep its idempotency promise.
- The phone's coupon and waiver fields need C2's columns to be stored.
- B1 is independent of everything else.
- Applying any of them is the founder's decision. This audit only records the dependencies.

01 — Database

Scope. VINII Partner, the Flutter mobile POS in `vinii_partner/`, at commit `da0a823`. Read-only audit, 2026-10-02. Nothing was changed.

Evidence labels

- **[code]**: read in this repository's source.
- **[ledger]**: read in a migration file.
  - `Repo-1-/supabase/migrations/` holds the OS and POS tables.
  - `OL-ui/db/migrations/` holds One Local's tables.
- **[snapshot]**: matches the production column list in `OL-ui/docs/CENTRAL_DB_MAP.md`, taken 2026-09-22.
- **[unverified]**: could not be checked.

This app has no database of its own. It has no migrations, no SQL files, no local SQLite, no Drift and no sqflite (`vinii_partner/pubspec.yaml`). It is a client of the shared Supabase project.

The only SQL in the repository is local-development tooling:

- `vinii_partner/tool/local_db/emulate_production_rls.sql` (and its `_undo`) copies production's member policies onto a local Docker replica.
- It is never run against production. Its header says so (lines 1–15).

1. Tables the app reads or writes directly (signed-in staff session, publishable key)

Table	Defined in	App access	Code
`organizations`	[ledger] Repo-1- 0001 (+ `theme` in 0009) · [snapshot]	read, through the `org_users` embed: `id, name, slug, type, status, theme, settings`	`lib/data/live/live_workspace.dart:32-37`
`org_users`	[ledger] 0004 · [snapshot]	read; **update** `role_id`, `status`	`live_workspace.dart:32`, `live_team.dart:26,68-73`
`roles`	[ledger] 0004 (rows seeded in 0004 and 0019) · [snapshot]	read	`live_team.dart:45`, embedded in `live_workspace.dart:35`
`org_modules`	[ledger] 0007 · [snapshot]	read `module_key, enabled`	`live_workspace.dart:86-89`
`role_module_access`	[ledger] 0008 · [snapshot]	read `organization_id, module_key, visible, roles!inner(slug)`	`live_workspace.dart:101-107`
`menu_categories`	[ledger] 0006	read; **insert, update, delete**	`live_menu.dart:30,64-79,90-95,111-115`
`menu_items`	[ledger] 0006 (+ columns in 0010)	read; **insert, update** (including `is_available`)	`live_menu.dart:35,154-165,175-180`
`dining_tables`	[ledger] 0006 (+ `area_id`, `qr_token`, `min_capacity` in 0010)	read	`live_pos.dart:31-37`, `live_tables.dart:24-30`
`dining_areas`	[ledger] 0010	read (embedded)	`live_pos.dart:33`, `live_tables.dart:28`
`orders`	[ledger] 0006 + 0010 · [snapshot]	read only	`live_pos.dart:38-43, 189-196,206-211`
`order_items`	[ledger] 0006 + 0010 · [snapshot]	read only (embedded in `orders`)	`live_pos.dart:24`
`payments`	[ledger] 0010 · [snapshot]	read only (embedded `amount`)	`live_pos.dart:24`
`org_bill_settings`	[ledger] 0010 · [snapshot]	read; **upsert** (ignore duplicates); **update** guarded by `updated_at`	`live_pos.dart:69-73`, `live_bill_settings.dart:26-67`

Table	Defined in	App access	Code
`channel_integrations`	[ledger] 0010	read; **update** `is_accepting`, `next_open_at`	`live_channels.dart:24–51`
`vinii_storefronts`	**draft only, not applied**: Repo-1- branch `feat/experience-cms`, `0021_vinii_storefronts.sql`	read; **upsert** `draft`, `updated_by`; **update** `draft = null`	`live_storefront.dart:28–32,81–85,99–102`
`vinii_storefront_versions`	**draft only, not applied**: Repo-1- branch `feat/experience-cms`, `0021_vinii_storefronts.sql`	read	`live_storefront.dart:33–38`

Storage: bucket `item-images` ([ledger] 0006; [snapshot] "Storage bucket: item-images (public)").

- The app uploads to `<organization_id>/<itemId>-<epoch ms>.<ext>` with `upsert: false` and stores the public URL in `menu_items.image_url` (`live_menu.dart:213–222`).
- Files are never deleted.

2. Tables written on the phone's behalf by One Local's server (service role)

The Orders tab calls One Local's partner API (`/api/partner/*`, §5 of `07_API_RPC_MAP.md`). That server runs database functions with the service role. The phone never touches these tables itself.

Table	Defined in	Written by
`ol_orders`	OL-ui 0021, columns added in 0022	`ol_create_restaurant_order` (test order), `ol_partner_act`
`ol_merchant_channels`	OL-ui 0022	`PATCH /outlet` (server code), `ol_partner_act` (`missed_in_a_row`, auto-pause)
`orders`, `order_items`, `kot_tickets`, `order_events`, `payments`	Repo-1- 0006, 0010	`ol_create_restaurant_order`, `ol_partner_act`
`org_counters`	Repo-1- 0006	`next_org_seq()` (order and KOT numbers)
`delivery_jobs` (and related)	OL-ui 0029	One Local `server/dispatch.ts` on accept or reject. [unverified] whether 0029 is applied in production

3. Columns of the POS transaction tables

`orders` ([ledger] 0006 + 0010; [snapshot] identical column list)

Column	Type / constraint
`id`	uuid PK, `gen_random_uuid()`
`organization_id`	uuid not null → `organizations(id)` on delete cascade
`order_no`	bigint not null; **unique (`organization_id`, `order_no`)**
`display_no`	text not null
`channel`	text, default `dine_in`, check in (`dine_in`, `delivery`, `pickup`, `online`, `swiggy`, `zomato`, `other`)
`status`	text, default `new`, check in (`new`, `in_billing`, `sent_to_kitchen`, `ready`, `awaiting_payment`, `paid`, `delivered`, `cancelled`) (0010 replaced the 0006 check)
`table_id`	uuid → `dining_tables(id)` on delete set null
`customer_name`, `customer_phone`, `captain_name`	text
`subtotal`, `gst_amount`, `total`	numeric(10,2) default 0
`gst_pct`	numeric(4,2) default 5
`payment_method`	text, null or in (`cash`, `upi`, `card`, `split`)
`created_by`	uuid → `auth.users` on delete set null

Column	Type / constraint
`created_at`, `updated_at`	timestamptz; trigger `orders_set_updated_at`
`session_id`	uuid → `table_sessions(id)` on delete set null (0010)
`placed_by`	text not null default `pos`, check in (`pos`, `captain`, `guest`, `integration`) (0010)
`guest_count`	int default 1 (0010)
`discount`, `service_charge`, `round_off`	numeric(10,2) default 0 (0010)
`paid_at`	timestamptz (0010)
`notes`, `external_ref`	text (0010). `external_ref` has **no unique constraint**

- **Indexes:** `orders_organization_id_idx`, `orders_org_status_idx (organization_id, status)`, `orders_table_id_idx`, `orders_created_at_idx (created_at desc)`, `orders_session_id_idx`.
- **There is no** `location_id`, customer id, device id, idempotency key or payment-status column.

`order_items` ([ledger] 0006 + 0010; [snapshot])

- **Base columns (0006):** `id` uuid PK; `order_id` → `orders` on delete cascade; `menu_item_id` → `menu_items` on delete set null; `name` text; `qty` int check > 0; `unit_price` numeric check ≥ 0; `add_ons` jsonb default `[]`; `notes`; `created_at`.
- **Added in 0010:** `variant_name`; `status` text default `pending`, check in (`pending`, `preparing`, `ready`, `delivered`, `cancelled`); `kot_ticket_id` → `kot_tickets` on delete set null; `course` default `eat`; `station` default `main`.
- **Indexes:** `order_id`, `menu_item_id`, `kot_ticket_id`.
- **No `organization_id`.** Row security goes through the parent order.

`kot_tickets` ([ledger] 0006; [snapshot])

- `id`; `organization_id` → orgs (cascade); `order_id` → `orders` (cascade).
- `kot_no` bigint, **unique (`organization_id`, `kot_no`)**.
- `station` default `main`.
- `status` check in (`new`, `preparing`, `ready`, `delivered`). There is **no `cancelled` value**.
- `priority` check in (`normal`, `urgent`).
- `created_at`, `updated_at`, with trigger `kot_tickets_set_updated_at`.
- **Indexes:** `organization_id`, `order_id`, `(organization_id, status)`.

`payments` ([ledger] 0010; [snapshot])

- `id`; `organization_id` (cascade); `order_id` → `orders` (cascade).
- `method` check in (`cash`, `upi`, `card`, `wallet`, `due`, `other`).
- `amount` numeric check **> 0**.
- `reference` text; `created_by` → `auth.users`; `created_at`.
- **Indexes:** `order_id`, `(organization_id, created_at desc)`.
- **No idempotency key, no status, no unique reference.**

`order_events` ([ledger] 0010; [snapshot])

- `id`; `organization_id`; `order_id` (cascade); `kind` text **with no check**; `message` text not null; `actor` text; `meta` jsonb; `created_at`.
- **Indexes:** `(order_id, created_at)`, `(organization_id, created_at desc)`.
- The comment calls it "append-only", but nothing enforces that: there is no trigger, and an update policy exists through `apply_org_rls`.

`org_counters` ([ledger] 0006; [snapshot])

- PK (`organization_id`, `kind`); `value` bigint.
- Only changed through `next_org_seq(org, kind)`, which is security definer and atomic: `insert … on conflict do update set value = value + 1`.
- Kinds in use: `order`, `kot`.

Menu, tables and settings

- **`menu_categories` (0006):** `id`, `organization_id`, `name` (not blank), `sort_order`, `created_at`. Index on `organization_id`. No `updated_at` and no trigger.
- **`menu_items`:**
  - 0006: `id`, `organization_id`, `category_id` → `menu_categories` on delete set null, `name` (not blank), `sku`, `price` numeric ≥ 0, `image_url`, `is_available` default true, `sort_order`, `created_at`, `updated_at` (trigger).
  - Added in 0010: `description`, `food_type` check in (`veg`, `non_veg`, `egg`), `course` check in (`eat`, `drink`), `is_recommended`, `prep_minutes` default 10, `station` default `main`.
- **`dining_tables`:**
  - 0006: `id`, `organization_id`, `label` with unique (org, label), `seats` > 0, `is_active`, `sort_order`, `created_at`.
  - Added in 0010: `area_id` → `dining_areas` set null, `qr_token` **globally unique**, `min_capacity`.
- **`dining_areas` (0010):** `id`, `organization_id`, `name` with unique (org, name), `sort_order`, `created_at`.
- **`org_bill_settings` (0010):**
  - Columns: PK `organization_id` → orgs; `legal_name`, `address`, `gstin`, `fssai`, `phone`; `sgst_pct` and `cgst_pct` (default 2.5 each); `service_charge_pct` (default 0); `round_off_enabled`; `footer_note`; `updated_at`.
  - Trigger `org_bill_settings_set_updated_at`.
  - Check: each percentage between 0 and 50.
- **`channel_integrations` (0010):**
  - Columns: `id`, `organization_id`, `channel` check in (`swiggy`, `zomato`, `ondc`, `website`, `whatsapp`, `qr`), unique (org, channel); `display_name`, `is_connected`, `is_accepting`, `next_open_at`, `auto_accept`, `commission_pct`, `config`, `updated_at`.
  - Trigger on `updated_at`.

Kernel

- **`organizations` (0001 + 0009):** `id`, `name`, `slug` (unique index), `type` **enum `org_type` (`franchise`, `investor`)**, `status` **enum `org_status` (`onboarding`, `active`, `suspended`)**, `legal_name`, `gstin`, contact fields, `settings` jsonb, `theme` jsonb (0009), `created_by`, `created_at`, `updated_at` (trigger). [snapshot] adds `source_lead_id`, `onboarding_stage`, `onboarding_owner` and `lifecycle_stage`.
- **`org_users` (0004):** `id`, `organization_id`, `user_id` → `auth.users`, `role_id` → `roles`, `email`, `full_name`, `status` **enum `member_status` (`active`, `suspended`)**, timestamps (trigger). **Unique (`organization_id`, `user_id`)**: one person may belong to several organizations.
- **`roles` (0004):** `id`, `slug` unique, check `^[a-z0-9]+(_[a-z0-9]+)*$`, `name`, `description`, `is_system`, `created_at`.
- **`org_modules` (0007):** PK (`organization_id`, `module_key` → `modules`), `enabled`, `config`, timestamps (trigger). The table comment says "Absence of a row means the module is not available".
- **`role_module_access` (0008):** `id`, `organization_id` (nullable; null = platform default), `role_id`, `module_key`, `visible`; unique nulls not distinct (org, role, module).

4. Enums

There are three Postgres enums: `org_type`, `org_status` and `member_status` (above). Every other status on a POS table is `text` with a `check` list.

5. Triggers

Only `set_updated_at` triggers exist on the tables above. They are on:

- `organizations`, `org_users`, `org_modules`, `role_module_access`
- `menu_items`, `orders`, `kot_tickets`
- `org_bill_settings`, `channel_integrations`, `item_channel_status`
- `ol_merchant_channels`

There is **no** trigger on `menu_categories`, `dining_tables`, `order_items`, `payments` or `order_events`.

There are **no business triggers**:

- no status-transition guard

- no totals check
- no stock or inventory trigger
- no event-emitting trigger

6. Functions

Function	Defined in	Called by this app?
`set_updated_at()`	Repo-1- 0001	trigger only
`current_org_id()`	Repo-1- 0004: `select organization_id from org_users where user_id = auth.uid() and status = 'active' limit 1`	indirectly, through every policy
`is_org_admin(org)`	Repo-1- 0004: role slug = `org_admin` only	indirectly (`org_users` writes)
`is_platform_admin()`	Repo-1- 0001	indirectly
`next_org_seq(org, kind)`	Repo-1- 0006	no (the server functions call it)
`apply_org_rls(regclass)`	Repo-1- 0010 (helper that creates policies)	no
`ol_create_restaurant_order(p jsonb)`	OL-ui 0023 (service role only)	indirectly: `POST /api/partner/test-order`
`ol_partner_act(order, action, actor, actor_name, expected, prep, reason)`	OL-ui 0023 (service role only)	indirectly: accept, reject, ready, out, delivered
`ol_expire_unanswered(org)`	OL-ui 0023 (service role only)	indirectly: every `GET /api/partner/orders`
`pos_place_order(p)`	**none in any ledger**	**yes**, `live_pos.dart:100`
`pos_record_payment(p)`	**none**	**yes**, `live_pos.dart:156`
`pos_set_order_status(p)`	**none**	defined in `live_pos.dart:225` but **no screen calls it**
`pos_adjust_bill(p)`	**none**	**yes**, `live_pos.dart:254`
`pos_issue_table_qr(p)`	**none**	**yes**, `live_tables.dart:76`
`vinii_publish_storefront(p)`	**draft only, not applied** (Repo-1- branch `feat/experience-cms`, `0021_vinii_storefronts.sql`)	**yes**, `live_storefront.dart:117`

No migration on any branch of Repo-1-, OL-ui or proviyaa-pos defines the five `pos_*` functions. This was confirmed by searching every `.sql` file on every branch (corrected 2026-10-02).

`vinii_publish_storefront` and the two storefront tables are defined in **one draft migration that is not applied**: `Repo-1-/supabase/migrations/0021_vinii_storefronts.sql`, on branch `feat/experience-cms` (worktree `Repo-1--experience-cms`). Its header says "DRAFT, NOT APPLIED". The first version of this audit missed it, because it searched only the checked-out branches. The full register is in `15_MIGRATIONS_NOT_APPLIED.md`.

Against a database built from these ledgers, each call fails with PostgREST `PGRST202` or Postgres `42883`. The app maps that to `FailureKind.notReady` and shows the "Not available yet" dialog (`live/errors.dart:33-41`). [unverified] Production's function list after 2026-09-22: the snapshot reports 29 functions but does not name them.

7. Views

None. Neither ledger contains a `create view` or `create materialized view`.

8. Realtime publication ([ledger])

- `supabase_realtime` includes `orders` and `kot_tickets` (0006: 448-449).
- It also includes `order_items`, `order_events`, `service_requests` and `table_sessions` (0010: 397-400).
- **The app subscribes to none of them.** See `08_REALTIME.md`.

9. Present in the database, not used by this app

These exist in the ledgers but the app never reads or writes them directly:

- `kot_tickets`, `order_events`, `org_counters`
- `table_sessions`, `service_requests`
- `menu_item_variants`, `kitchen_stations`, `item_channel_status`
- `locations` (0018; 0 rows in the snapshot), `org_invites`

Variants are not read anywhere, so a menu item's variants are invisible to the phone.

02 — Order flow

Scope. `vinii_partner/` at `da0a823`. Evidence labels are as in `01_DATABASE.md`. Paths are relative to `vinii_partner/lib/` unless they say otherwise.

The app has **three separate order flows**. They share no code path for writing.

Flow	Screens	Who writes the database
A. Phone POS (dine-in, takeaway, delivery taken at the counter)	Sell → Checkout → Order	Would be the RPC `pos_place_order`, which **does not exist** in any migration (01 §6)
B. Guest on a handed-over phone	Guest runtime (`/guest`)	The same RPC as A, with `placed_by = guest`
C. Marketplace delivery (One Local)	Orders tab, new-order alert	One Local's server, through `ol_partner_act` (exists, OL-ui 0023)

A. Phone POS

A1. LOGIN

- **Screen:** `presentation/auth/sign_in_screen.dart` (route `/sign-in`).
- **User action:** enters an email and password, then taps Sign in.
- **Code:** `SessionController.signIn` (`application/session_controller.dart:119`) → `LiveAuth.signIn` (`data/live/live_auth.dart:28–48`).
- **API:** Supabase Auth `signInWithPassword`; the client uses the PKCE flow (`live_backend.dart:89–96`).
- **Tables:** `auth.users` (Supabase-managed; the app writes nothing).
- **Insert / update:** none by the app.
- **Status:** the session stage (`SessionStage`, `session_controller.dart:14–35`) moves from `signedOut` to `noAccess`, `choosingOrg`, `updateRequired` or `ready`. A failure goes to `bootError`.
- **Side effect:** supabase_flutter persists the session.
- **Then, in `_loadWorkspace` (`session_controller.dart:132–158`):**
  - It reads `org_users` with `roles` and `organizations` (`live_workspace.dart:32–37`).
  - It writes `last_org:<userId>` to SharedPreferences.
  - It reads `org_modules` and `role_module_access` (`live_workspace.dart:82–125`).
  - If the app version is below `organizations.settings.app.min_app_version`, it opens the update-required gate (`:153`).
- **Event:** none.

A2. LOCATION: not implemented

The app has no location or branch step and no `location_id` anywhere. The step it does have is **organization selection**:

- It opens the remembered organization, or the only one, automatically.
- Otherwise it shows `OrgPickerScreen` (`/organizations`).
- `WorkspaceRepository.activate` only sets `activeOrganization` in memory (`live_workspace.dart:77–79`). Nothing is written to the database.
- **Database limit:** `org_users_member_select` filters by `current_org_id()`, which returns one membership (`limit 1`). A person in two organizations therefore sees only one of them in the picker. The code acknowledges this at `live_workspace.dart:28–31`.

A3. TABLE / ORDER TYPE

- **Screens:** `presentation/pos/sell_screen.dart` and `presentation/pos/ticket_view.dart`. TicketView sits beside the menu on wide screens and on `/sell/checkout` on phones (`checkout_screen.dart:29`).
- **User action:** picks Dine-in, Takeaway or Delivery, then picks a table for Dine-in.
- **Code:** `SellCart.setServiceType`, `setTable`, `setGuestCount` (`application/sell_cart.dart:194–208`).
- **API, for the table list:** `LivePos.tables()` (`live_pos.dart:27–64`) runs two reads:
  - `dining_tables` where `is_active`, ordered by `sort_order` and `label`, with `dining_areas.name`.
  - `orders.table_id` where `status not in (paid, cancelled)`, to mark tables as occupied.

- **Insert / update:** none. **Status:** none.
- **Mapping:** Dine-in → `dine_in`, Takeaway → `pickup`, Delivery → `delivery` (`domain/pos_order.dart:4–13`).

A4. CUSTOMER

- **Screen:** TicketView text fields.
- **User action:** types a name and phone. These only matter for takeaway and delivery; for dine-in they are not sent (`sell_cart.dart:278–279`).
- **Code:** `SellCart.setCustomerName` and `setCustomerPhone`. A delivery order is blocked until the phone has at least 10 digits (`sell_cart.dart:255–258`).
- **Tables:** none. **There is no customer entity or customer ID.** The name and phone travel as free text in the RPC payload, as `customer_name` and `customer_phone`.

A5. PRODUCT

- **Screen:** `sell_screen.dart`, a menu grid with category chips.
- **User action:** taps Add, or long-presses to set quantity and a note.
- **Code:** `SellCart.load` (`sell_cart.dart:71–92`) → `LiveMenu.load` (`live_menu.dart:25–48`) and `LivePos.billCharges` (`live_pos.dart:67–83`).
- **Tables read:**
  - `menu_categories`
  - `menu_items`
  - `org_bill_settings`, for `sgst_pct`, `cgst_pct`, `service_charge_pct` and `round_off_enabled`. Defaults are used when the table or row cannot be read.
- **Rules:**
  - An item with `is_available = false` cannot be added (`sell_cart.dart:124`).
  - Variants (`menu_item_variants`) are never read.
  - Per-channel availability (`item_channel_status`) is not consulted.

A6. CART

- **State:** `SellCart`, a `ChangeNotifier` held **in memory only** and not persisted. It is discarded on sign-out and on organization switch.
- **User actions:** change quantity, add a note, clear the order (`ticket_view.dart _clear`), and remove the service charge before sending (`setServiceChargeWaived`, for owner, admin and manager).
- **Bill:** `BillSummary.of(...)` gives an **estimate** only (`sell_cart.dart:290–306`). Its own comment says the server prices the order.
- **Tables:** none.

A7. ORDER (the only write in the POS flow)

- **Screen:** TicketView, with two buttons: **Save & hold** (`_send(fire: false)`) and **Send to kitchen** (`_send(fire: true)`) (`ticket_view.dart:445–460`).
- **Code path:**
  - `_send` sets `placed_by` to `captain` when the role slug is `captain`, and `pos` otherwise (`ticket_view.dart:77–81`).
  - `SellCart.place` (`sell_cart.dart:317–345`).
  - `PendingKeys.run('order:<orgId>')` (`application/sell_keys.dart:36–49`). The idempotency key is a UUID v4, **saved to SharedPreferences before the request is sent**.
  - `LivePos.placeOrder` (`live_pos.dart:86–139`).
- **RPC:** `pos_place_order(p)` with:
  - `organization_id`, `idempotency_key`, `service_type`
  - `table_id`, `guest_count`, `customer_name`, `customer_phone`, `notes`, `scheduled_for`
  - `placed_by`, `fire_kot`
  - `waive_service_charge` and `waive_reason` (only when waived)
  - `coupon_code` (only when set)
  - `lines[{menu_item_id, qty, notes}]`
  - **No prices, totals or variants are sent.**

- **Tables, insert, update and status:** whatever the server function does. The function **is not defined in any migration**, so on a database built from the ledgers there is **no write at all**.
- **Expected response:** an order row (`live_pos.dart:287–348`) with `id`, `display_no`, `status`, `channel`, `updated_at`, the money fields, `lines` or `order_items`, and `paid`.
- **Today's actual outcome:**
  - PostgREST `PGRST202` or `42883` becomes `FailureKind.notReady`.
  - A dialog says "Taking orders from the phone needs the POS order functions … take this order at the POS counter" (`live_pos.dart:16–18`).
  - The key **stays saved**: only `rejected` and `forbidden` clear it (`sell_keys.dart:44–46`).
- **On success:**
  - If the returned lines match the cart, the cart is cleared and the app opens `/sell/order/<id>`.
  - If they differ, the server returned an earlier order for that key. The dialog "An earlier order went through" appears and the cart is kept (`ticket_view.dart:116–142`).
- **Event:** none by the app. The contract expects the server to write `order_events` (`placed`, `kot_fired`); not verifiable.

A8. ORDER ITEMS: not written by the app

The lines exist only inside the RPC payload. The app reads `order_items` back, embedded in `orders` (`live_pos.dart:24`), and **hides lines whose status is `cancelled`** (`live_pos.dart:292`).

A9. KOT → KITCHEN → READY: not implemented for phone POS orders

- The app has no kitchen display and no "mark ready" for POS orders.
- `PosRepository.setStatus` → `pos_set_order_status` exists (`live_pos.dart:218–245`), but **no screen or controller calls it**. A search of `presentation/` and `application/` found only `TeamRepository.setStatus` callers.
- The only KOT intent is the `fire_kot` flag in A7. See `03_KOT_FLOW.md`.

A10. BILL

- **Screen:** `presentation/pos/pos_order_screen.dart` (`/sell/order/<id>`).
- **Reads:** `LivePos.refresh` (`live_pos.dart:203–216`) fetches `orders` with `dining_tables`, `order_items` and `payments(amount)`. The screen shows the **stored** figures (`subtotal`, `discount`, `service_charge`, `gst_amount`, `round_off`, `total`) and never recomputes them (`live_pos.dart:329–341`).
- **User action:** remove the service charge from a saved, unpaid order, or put it back.
- **Code:** `_adjust` (`pos_order_screen.dart:210–240`) → `LivePos.adjustBill` (`live_pos.dart:247–275`).
- **RPC:** `pos_adjust_bill(p {order_id, expected_updated_at, waive_service_charge, waive_reason})`. **It does not exist**, so the app shows "Not available yet".
- **Local side effect:** the cashier's keep-or-remove answer to the final-order prompt is remembered in SharedPreferences (`service_charge_choice:<orderId>`, `local_prefs.dart:59–62`).
- **No bill printing exists** (see `09_HARDWARE.md`).

A11. PAYMENT

Done in `presentation/pos/payment_card.dart`, which calls `pos_record_payment`. That function **does not exist**. Full detail is in `04_PAYMENT_FLOW.md`.

A12. COMPLETION: no separate step

- The app treats an order as finished when the server returns `status = paid`, or when the amount due is at most 0.009 (`domain/pos_order.dart`, `PlacedOrder.isPaid`).
- The "Open orders" sheet (`presentation/pos/open_orders_sheet.dart:48`) lists **today's orders not in (`paid`, `cancelled`)**, newest first, up to 100 (`live_pos.dart:180–201`).
- Completed or cancelled orders cannot be reopened from the phone.

B. Guest on a handed-over phone

- **Entry:** the Sell screen's hand-to-guest button, for a chosen table (`sell_screen.dart:84–85` → `/guest?table=<id>`).
- **Code:**
  - `GuestSession.place` (`application/guest_session.dart:435–470`) builds a `PosOrderDraft` with `placed_by = guest`.

- Delivery details are folded into `notes` as `"Deliver to: …"` (`:425–431`).
- It sends through `PosGuestGateway.place` (`application/guest_gateway.dart:60–79`), which calls the same `pos_place_order` as A7, **under the staff member's session**.

- **Idempotency key:** UUID v4 held **in memory only**, in `GuestSession.pendingKey` (`guest_session.dart:437`). Unlike A7 it is **not saved on the phone**, so killing the app loses it.
- **Tracking:** `PosGuestGateway.watch` polls `LivePos.refresh` every 5 s while the order is open (`guest_gateway.dart:82–94`).
- **Studio preview:** `PreviewGuestGateway` returns a local stand-in order numbered `PREVIEW` and writes nothing (`guest_gateway.dart:100–127`).

C. Marketplace delivery orders (One Local)

The phone only calls One Local's HTTP API (`data/live/partner_api_inbox.dart`). One Local's server code is in `OL-ui/frontend/server/restaurant.ts` and `partner.ts`. It runs the database functions with the service role. Function bodies were read in `OL-ui/db/migrations/0023_partner_order_functions.sql`.

Step	Screen / user action	Phone call	Server and database effect ([ledger] OL-ui 0023 and `restaurant.ts`)	Event rows
Order arrives	none: a poll every 5 s while accepting, every 30 s otherwise (`application/inbox_controller.dart:77–88`)	`GET /api/partner/orders?tab=new`	First runs `ol_expire_unanswered(org)` (`restaurant.ts:49`), which can **cancel** orders past their deadline (C7). Then reads `ol_orders` joined to `orders` and `order_items`	none on a plain read
Alert	`presentation/orders/new_order_alert_screen.dart` rings and vibrates (`application/ring_player.dart`)	—	—	—
Accept	Accept with a prep time (`new_order_alert_screen.dart:173`, `delivery_order_screen.dart:181`)	`POST /orders/<posOrderId>/accept {version, prep_minutes}`	**Checks:** role (`require(staff,'accept')`), organization (`restaurant.ts:81–82`), version (`OL_STALE`). **Writes:** `kot_no = next_org_seq(org,'kot')`; **INSERT** `kot_tickets` (station `main`, status `new`); **UPDATE** `order_items.kot_ticket_id` for pending lines; **UPDATE** `orders.status` `new` → `sent_to_kitchen`; **UPDATE** `ol_orders` (`accepted`, `prep_minutes`, `eta_minutes`); **UPDATE** `ol_merchant_channels.missed_in_a_row = 0`. **Then:** dispatch `onRestaurantAccepted` touches `delivery_jobs`	`accepted`, `kot_fired`
Reject	Reject with a reason (`new_order_alert_screen.dart:181`, `delivery_order_screen.dart:92`)	`POST …/reject {version, reason}`	`orders` → `cancelled`; `order_items` → `cancelled`; `ol_orders` → `cancelled` with `cancel_reason`; missed counter reset; dispatch `onRestaurantRejected`	`rejected`
Ready	Mark ready (`delivery_order_screen.dart:207`)	`POST …/ready {version}`	`kot_tickets` → `ready`; `order_items` → `ready`; `orders` → `ready`; `ol_orders` → `packed`	`kot_ready`
Out	Slide to confirm (`delivery_order_screen.dart:216`)	`POST …/out {version}`	`kot_tickets` → `delivered`; `order_items` → `delivered`; `orders.updated_at` bumped (**status stays `ready`**); `ol_orders` → `out`	`out_for_delivery`
Delivered	Slide to confirm (`delivery_order_screen.dart:226`)	`POST …/delivered {version}`	`orders` → `delivered`, with `payment_method = cash` and `paid_at`; **INSERT** `payments` (cash, `amount = orders.total`, `reference = 'onelocal:<ol_orders.id>'`); `ol_orders` → `delivered` with `payment_status = paid`	`delivered`, `paid`
Time-out	none (server side)	(any list call)	`ol_partner_act(…,'timeout')`: orders and items → `cancelled`; `ol_orders` → `cancelled` with "did not respond in time"; `missed_in_a_row + 1`; **auto-pause** (`accepting = false`) at `auto_pause_after`	`timed_out`
Outlet on/off	Outlet bar (`presentation/orders/outlet_bar.dart:69`)	`PATCH /outlet {accepting, back_at?}`	Updates `ol_merchant_channels` (`restaurant.ts:105–115`)	none
Test order	Settings → Send test order	`POST /test-order`	`ol_create_restaurant_order` with 1–2 random available items, **no idempotency key**: INSERT `orders` (`channel online`, `placed_by integration`, `display_no 'OL-'` + 5 digits, GST fixed at 5%), `order_items`, `order_events`, `ol_orders`; then a push notification to `ol_partner_devices` (`restaurant.ts:177–191`)	`placed`

Status words. The phone shows One Local's status words (`placed`, `accepted`, `packed`, `out`, `delivered`, `cancelled`) and receives the POS word separately as `pos_status` (`domain/delivery_order.dart`, `restaurant.ts:26–31`).

Version check. The `version` the phone sends back is `orders.updated_at`.

Push notifications. This app **never registers a device**: it never calls `PUT /api/partner/devices`. So One Local's push for new orders cannot reach it. New orders are found only by polling while the app is open.

03 — KOT (kitchen order ticket) flow

Scope. `vinii_partner/` at `da0a823`. Evidence labels are as in `01_DATABASE.md`.

Summary.

- **This app never reads or writes `kot_tickets` directly.** No `.from('kot_tickets')` appears anywhere in `lib/`.
- KOTs come from two places:
  - **Marketplace (One Local) orders:** One Local's `ol_partner_act` creates and advances them when the phone accepts, marks ready or sends out an order. This exists ([ledger] OL-ui 0023).
  - **Phone POS orders:** the `fire_kot: true` flag sent to `pos_place_order`. That function **does not exist** in any migration, so no KOT is created today.

Creation

Path	Trigger	What is written	Evidence
One Local order accepted	Accept on the phone → `POST /api/partner/orders/<id>/accept` → `ol_partner_act(…,'accept')`	Number from `next_org_seq(org,'kot')`; **INSERT** `kot_tickets (organization_id, order_id, kot_no, station = 'main', status = 'new')`; **UPDATE** `order_items` set `kot_ticket_id` where `order_id` matches and `status = 'pending'`; **INSERT** `order_events` `kot_fired` ("KOT n fired for …")	OL-ui 0023, accept branch
Phone POS "Send to kitchen"	`ticket_view.dart:458` → `SellCart.place( fireKitchenTicket: true)` → `pos_place_order(p.fire_kot = true)`	Unknown: the function is not defined	`live_pos.dart:114`; 01 §6
Phone POS "Save & hold"	`ticket_view.dart:445` → `fire_kot: false`	Intended: no KOT	`live_pos.dart:114`

Not in this audit: the Windows POS repository (`proviyaa-pos`) and the web counter in Repo-1- may also create KOTs. Repo-1-'s `apps/web/src/lib/pos.ts` contains `setKotStatus`. Neither is part of this app.

Numbering

- `kot_no` is a per-organization running number from `org_counters` (kind `kot`), through the security-definer function `next_org_seq`. It is atomic (`insert … on conflict do update value + 1`) ([ledger] Repo-1- 0006).
- `kot_tickets` has **unique (`organization_id, kot_no`)**.
- There is no display prefix. The event message uses the bare number.

Order → KOT relationship

- `kot_tickets.order_id` → `orders(id)` on delete cascade. **One order can have many KOTs**: there is no unique rule on `order_id`.
- Today's only creator (accept) makes **one KOT per accept**. Accept is allowed only from `new`, so in practice there is one KOT per One Local order.

KOT ↔ item relationship

- `order_items.kot_ticket_id` → `kot_tickets(id)` on delete set null (0010). It is set only for lines in `pending` status at accept time.
- `kot_tickets.station` and `order_items.station` default to `main`. `kitchen_stations` exists (0010), but nothing in this flow routes lines to stations: accept always uses `'main'`.

Kitchen status

- **Allowed values** (`kot_tickets.status`): `new` → `preparing` → `ready` → `delivered`.
- **Who sets which value** (in the code read for this audit):

Value	Set by
`new`	`ol_partner_act` accept
`preparing`	**nothing**. Neither this app nor `ol_partner_act` sets it. The event kind `kot_preparing` is shown as "Kitchen started" in the phone's timeline labels (`domain/delivery_order.dart`), but no writer exists in either codebase
`ready`	`ol_partner_act` ready, which updates tickets in (`new`, `preparing`)
`delivered`	`ol_partner_act` out, which updates **all** tickets of the order

- **Order-line status runs alongside:** `order_items.status` goes `pending` → `ready` (on ready) → `delivered` (on out), or `cancelled`.
- **The phone has no kitchen display.** It cannot set `preparing` and cannot mark a phone POS order ready (no caller of `setStatus`, see 02 §A9).

Ready

"Mark ready" exists only for One Local orders: `delivery_order_screen.dart:207` → `ol_partner_act(ready)`. It updates three tables:

- `kot_tickets` → `ready`
- `order_items` → `ready`
- `orders.status` → `ready`

It also sets `ol_orders.status` → `packed` and writes the event `kot_ready`.

Cancellation

- `kot_tickets.status` has **no `cancelled` value**.
- Reject and time-out happen only while `orders.status = 'new'`, which is **before** a KOT exists on the One Local path. They cancel `orders` and `order_items` and do not touch `kot_tickets`.
- No path cancels a KOT that has already fired: not in the app, and not in `ol_partner_act`.

Reprint

Not implemented. The app has no KOT or bill printing at all (see `09_HARDWARE.md`). The only printing is the table QR-code PDF sheet.

04 — Payment flow

Scope. `vinii_partner/` at `da0a823`. Evidence labels are as in `01_DATABASE.md`.

The app never inserts or updates `payments` itself. It reads `payments(amount)`, embedded in `orders`, to work out what has been paid (`live_pos.dart:24,301–306`).

Payments are written in two places:

Path	Writer	Exists?
Phone POS (counter)	RPC `pos_record_payment`	**No.** It is not defined in any migration (01 §6)
One Local delivery order, "Delivered"	One Local server → `ol_partner_act(…,'delivered')`	**Yes** ([ledger] OL-ui 0023)
Guest online payment (Cashfree)	A server seam, `GuestPayments`	**Not wired.** `PosGuestGateway.payments` returns `null` (`application/guest_gateway.dart:36`)

1. Phone POS payment

Screen: `presentation/pos/payment_card.dart`, inside `pos_order_screen.dart`.

User action

- Choose Cash, UPI, Card or Split.
- Enter an amount. For cash, also enter what the guest handed over. For UPI or card, optionally a reference.
- Tap Charge.

Code

- `_charge` (`payment_card.dart:184–225`)
- → `PendingKeys(prefs).run('payment:<orderId>', …)` (`application/sell_keys.dart`)
- → `LivePos.recordPayment` (`live_pos.dart:141–178`)

RPC

- `pos_record_payment(p {order_id, method, amount, idempotency_key, reference, expected_updated_at})`.
- `method` is one of `cash`, `upi`, `card` (`domain/pos_order.dart:51–59`).
- `expected_updated_at` is the order's `updated_at` as last read.

Client checks: amount > 0 (`live_pos.dart:149–154`). On a single payment, `_singleProblem` (`payment_card.dart:147–160`) also blocks an amount above what is due, and cash received below the amount.

Expected response: the order, including `paid` (sum of payments) and `status`.

Actual outcome today: "Not available yet": "*Recording payments from the phone needs the POS payment function … Settle this bill at the POS counter.*" The payment key stays saved.

Payment methods

- Offered by the phone: `cash`, `upi`, `card`.
- Allowed by the database (`payments.method`): `cash`, `upi`, `card`, `wallet`, `due`, `other`.
- Allowed on `orders.payment_method`: `cash`, `upi`, `card`, `split`.

Split payments

- **Code:** `_chargeSplit` (`payment_card.dart:227–262`) and `_secondPart` (`:265–289`).
- **How it works:**
  - The two parts must add up exactly to what is due. Short or over blocks the button.
  - Each part is a **separate `pos_record_payment` call** with its own saved key: `payment:<orderId>` and `payment:<orderId>:2`.
  - If part 2 fails, part 1 stays recorded, and the card offers "try the second part again" (`_retrySecondPart`, `:291–303`).
- In the database a split is simply several `payments` rows. `orders.payment_method = 'split'` exists, but this app never sends it.

Payment status

- **There is no payment-status column** on `orders` or `payments`.
- The phone derives it from the order:
  - `paid` = the RPC's `paid`, otherwise the sum of `payments.amount`.
  - `due` = `total − paid`.
  - `isPaid = status == paid`, or `due ≤ 0.009` (`domain/pos_order.dart:228`).

Transaction IDs

- **Database side:** `payments.id` (uuid, generated by the database) and `payments.reference` (free text, **not unique**).
- **Phone side:** the idempotency key (UUID v4) is generated on the phone. The `payments` table has no column to store it.
- The phone shows no gateway or terminal transaction id: there is no card terminal integration.

Cash handling

- The cash tendered and the change to give back are worked out **on the phone, for display only** (`payment_card.dart`, `cashReceived` passed to `onRecorded`).
- The cash tendered is **not sent** to the server: the RPC payload has no such field.
- There is no cash drawer, till session or shift in this app.

Refunds and settlement

None in this app. It has:

- no refund screen
- no refund RPC call
- no settlement or payout logic

One Local has `ol_refunds` and `ol_process_refund` (OL-ui 0024), but this app never calls them.

2. One Local delivery order: cash on delivery

Trigger: "Delivered" (slide to confirm, `delivery_order_screen.dart:226`) → `POST /api/partner/orders/<id>/delivered {version}`.

Server (`ol_partner_act`, delivered branch, [ledger] OL-ui 0023)

- **Check:** `ol_orders.status` must be `out`, otherwise `OL_WRONG_STATE`.
- **UPDATE `orders`:**
  - `status = 'delivered'`
  - `payment_method = 'cash'`
  - `paid_at = now()`
  - `updated_at = now()`
- **INSERT `payments`:**
  - `method = 'cash'`
  - `amount = orders.total`, which **includes the One Local delivery fee**. `ol_create_restaurant_order` sets `total = subtotal + gst + fee`.
  - `reference = 'onelocal:<ol_orders.id>'`
  - `created_by = actor`
- **UPDATE `ol_orders`:** `status = 'delivered'`, `payment_status = 'paid'`.
- **Events:** `delivered` and `paid` ("Cash ₹… collected on delivery").

What is not done

- No idempotency key protects this insert. A replay is blocked only by the state check (`out` is required), plus the `version` match.
- The final status is `delivered`, not `paid`.
- The partner API always reports `payment: 'Cash on delivery'` (`restaurant.ts`, `shape()`).

3. Guest online payment (Cashfree): present in code, not connected

- `application/guest_payments.dart` defines the flow. The server would create the gateway order and later report the verified outcome.
- `data/payments/cashfree_checkout_web.dart` loads Cashfree's JS SDK from its CDN and opens the checkout modal (sandbox by default).
- Non-web builds throw "Paying online opens in the web version" (`cashfree_checkout_stub.dart`).
- `GuestOrderGateway.payments` defaults to `null`, and `PosGuestGateway` does not override it. **No online payment can start from any shipped gateway.** The guest bill tells the guest to pay staff.

05 — Inventory flow

Scope. `vinii_partner/` at `da0a823`. Evidence labels are as in `01_DATABASE.md`.

Finding: inventory never changes

There is no inventory or stock data anywhere in the system this app uses:

- **[code]:** nothing in `vinii_partner/lib/` reads or writes a stock, inventory, recipe, ingredient, wastage, GRN or purchase table.
  - A search for `stock|inventor|recipe|ingredient` found only display text ("Item out of stock" as a reject reason), the role name `inventory_manager`, and the storefront's guest-facing ingredient list (marketing copy stored in the storefront definition, not stock).
- **[ledger]:** neither `Repo-1-/supabase/migrations` nor `OL-ui/db/migrations` creates any such table.
- **[snapshot]:** the 2026-09-22 production map (72 tables) contains no stock or inventory table.
- **Triggers:** none on `orders`, `order_items`, `kot_tickets` or `payments` other than `set_updated_at` (01 §5).

Relationship	Finding
ORDER → INVENTORY	None. Creating an order (a phone POS RPC, or One Local's `ol_create_restaurant_order`) writes no stock row
KOT → INVENTORY	None. Firing, readying or delivering a KOT writes no stock row
PAYMENT → INVENTORY	None

The nearest thing: availability switches (not quantities)

Write	Table / column	Where	Effect
Item sold-out switch	`menu_items.is_available`	`LiveMenu.setItemAvailable` (`live_menu.dart:172-190`); also the full item save (`saveItem`)	The item can no longer be added on the phone (`sell_cart.dart:124`). `ol_create_restaurant_order` also refuses it (`OL_ITEM_UNAVAILABLE`)
One Local channel availability	`item_channel_status` (channel `one_local`)	**Not written by this app.** One Local's partner items endpoint writes it (`restaurant.ts:136-155`), but this app does not call `/api/partner/items`	—

Both are on/off flags with no count. Nothing decrements them, and nothing switches an item off automatically when an order is placed.

06 — Offline and sync

Scope. `vinii_partner/` at `da0a823`. Evidence labels are as in `01_DATABASE.md`.

Summary: online-only, with no sync engine

The app cannot take an order, record a payment or change anything while offline. Every write is a live network call.

Item	Finding	Evidence
SQLite	**None.** No `sqflite`, `drift` or `powersync` dependency	`vinii_partner/pubspec.yaml` dependencies
Local tables	**None.** The only on-device store is SharedPreferences (key/value)	`lib/data/local_prefs.dart`
Local IDs	Only idempotency keys (UUID v4) and the request id. No local order, line or payment ids exist	below
Server IDs	Every entity id comes from the database (`gen_random_uuid()` defaults) or from the missing RPCs	01 §3
Outbox	**None.** Nothing queues a write for later	—
Sync	**None.** No background sync, no replication, no pull-on-reconnect	—
Retries	**Manual only.** A failed write shows an error and the user taps again. There is no automatic retry loop. The partner API call has a 15 s timeout (`partner_api_inbox.dart:69`)	—

What SharedPreferences holds (`lib/data/local_prefs.dart`)

Key	Purpose
`intro_seen`	first-launch intro shown
`last_org:<userId>`	the organization opened last
`theme_mode`, `ring_enabled`	device preferences
`pending_key:<scope>`	**idempotency keys waiting for a server answer**: `order:<orgId>`, `payment:<orderId>`, `payment:<orderId>:2`
`last_service_charge:<orgId>`	the last non-zero service charge percentage, for switching it back on
`service_charge_choice:<orderId>`	the cashier's keep-or-remove answer to the final-order prompt
`storefront_draft:<orgId>`	the storefront draft, **only when the `vinii_storefronts` table is missing** (`live_storefront.dart:79–93`)

The Supabase session is persisted by `supabase_flutter` itself (`local_prefs.dart:6–7` says so). This audit did not check where.

Idempotency (duplicate prevention for writes)

Write	Key	Lifetime	Reused on retry?	Evidence
Phone POS order (`pos_place_order`)	UUID v4, scope `order:<orgId>`	**Saved before the request is sent**; cleared on success, `rejected` or `forbidden`; **kept** on offline, not-ready or unknown errors	Yes, even after an app restart	`application/sell_keys.dart:36–49`
Phone POS payment (`pos_record_payment`)	UUID v4, scope `payment:<orderId>` (and `:2` for the second part of a split)	Same rules	Yes	`payment_card.dart:196–198,234–240,271–273`
Guest order on a handed-over phone	UUID v4 in `GuestSession.pendingKey`	**Memory only**; cleared on success	Yes, but only within the same app run	`guest_session.dart:437,460`
Partner API writes (accept, reject, ready, out, delivered, outlet, test order)	`idempotency-key` header, **a new UUID on every request**	per request	**No.** A retry sends a different key	`partner_api_inbox.dart:38–52`

Write	Key	Lifetime	Reused on retry?	Evidence
Every request	`x-request-id`, a new UUID per HTTP request	per request	No (tracing only)	`data/live/caller_client.dart:16-20`

Server side

- The `pos_*` functions that would honour the order and payment keys **do not exist** (01 §6).
- The `orders` and `payments` tables have **no column** to hold an idempotency key.
- One Local's partner API server **does not read** the `idempotency-key` header. A search of `OL-ui/frontend/server/{http,partner,restaurant}.ts` and `app/api/partner/` found no handling.
- One Local's own idempotency exists only for customer checkouts, on `ol_orders (customer_hash, idempotency_key)` in `ol_create_restaurant_order`. The partner test order sends no key.

Client-side double-send guards

- `SellCart.placing` (`sell_cart.dart:321-323`).
- `PaymentCard._paying`.
- `InboxController.busy` per order (`inbox_controller.dart:169-196`).

Conflict resolution (optimistic concurrency)

Write	Version token	On conflict
POS RPCs	`expected_updated_at` = `orders.updated_at` as last read	The server is expected to raise `POS_STALE`, which the app maps to `conflict` (`errors.dart:23-26`). Not verifiable: the functions are missing
One Local actions	`version` = `orders.updated_at`	`ol_partner_act` raises `OL_STALE`, the API answers 409, and the app shows a conflict and re-polls New (`inbox_controller.dart:184-190`)
Bill settings	`update … where organization_id = ? and updated_at = <loaded>` (`live_bill_settings.dart:58-67`)	Zero rows triggers a reload. A newer `updated_at` means conflict; otherwise it reports "not permitted". First save uses `upsert(ignoreDuplicates)`
Storefront publish	`expected_version`	The server is expected to raise `POS_STALE`. Not verifiable
Menu, channels, team	**none**: last write wins	—

What happens offline

Action	Behaviour
Boot, sign-in	Auth errors map to "You are offline …" (`errors.dart:144-148`); the session stage goes to `bootError` with a retry option
Sell → Send	The order **is not saved anywhere**. The error is shown, the in-memory cart stays, and the pending key stays in SharedPreferences, so the next tap reuses it
Payment	Not recorded; the key stays; tap again
Delivery inbox poll	The list keeps showing the last results; the failure is stored per tab; polling continues (`inbox_controller.dart:101-119`)
Guest tracking	Offline errors are swallowed and polling continues (`guest_gateway.dart:90-92`)
Storefront draft save	Kept on the phone **only** when the error is "table missing" (`notReady`). An offline error is thrown to the screen (`live_storefront.dart:87-92`)
Sign out	Always clears the local session, even offline (`live_auth.dart:69-75`)

07 — API and RPC map

Scope. `vinii_partner/` at `da0a823`. Evidence labels are as in `01_DATABASE.md`.

All network access is confined to `lib/data/`. A search of `lib/presentation/` and `lib/application/` for `.from(`, `.rpc(`, `.channel(`, `.stream(`, `storage.from`, `functions.invoke` and `http.` found no other database or network calls. The one exception is the Cashfree web SDK script loader (`data/payments/cashfree_checkout_web.dart`), which is not reachable today (04 §3).

Two transports are used:

1. The **Supabase client** (`supabase_flutter`), with the publishable key and the staff member's session.
2. **One Local's partner HTTP API**, at `PARTNER_API_BASE_URL` + `/api/partner`.

Both go through `CallerClient` (`data/live/caller_client.dart`), which adds these headers to **every** request:

- `x-app: vinii-partner`
- `x-app-version`, `x-platform`, `x-runtime`
- a fresh `x-request-id` (`live_backend.dart:48-55`)

1. Supabase Auth

Call	Code
`signInWithPassword(email, password)`, PKCE flow	`live_auth.dart:33`, `live_backend.dart:93-95`
`resetPasswordForEmail(email)`: errors are hidden except offline or rate-limit, so the reset form cannot reveal which accounts exist	`live_auth.dart:51-61`
`signOut(scope: local or global)`	`live_auth.dart:64-76`
`onAuthStateChange` (signed-out events only)	`live_auth.dart:23-25`
`currentSession.accessToken`, sent as the bearer token to the partner API	`partner_api_inbox.dart:39`

2. PostgREST table queries (every query filters `organization_id` explicitly, except the reads of `roles` and of a person's own `org_users` rows)

Repository	Method	Operation
`LiveWorkspace`	`memberships`	`GET org_users?user_id=eq.<uid>&select=status,role:roles(slug,name),organization:organizations(id,name,slug,type,status,theme,settings)`
	`appDefinition`	`GET org_modules?organization_id=eq.<org>`; `GET role_module_access?…roles!inner(slug)&role.slug=eq.<slug>&or=(organization_id.is.null,organization_id.eq.<org>)`
`LiveMenu`	`load`	`GET menu_categories`, `GET menu_items`
	`saveCategory`	`POST menu_categories` / `PATCH menu_categories?id&organization_id`
	`deleteCategory`	`DELETE menu_categories?id&organization_id` (returns ids; zero rows = error)
	`reorderCategories`	**one `PATCH` per category**, in sequence, not in one transaction (`live_menu.dart:108-120`)
	`saveItem`	`POST menu_items` / `PATCH menu_items?id&organization_id`
	`setItemAvailable`	`PATCH menu_items {is_available}`
`LivePos`	`tables`	`GET dining_tables` (+ `dining_areas.name`), `GET orders.table_id` where status is not in (`paid`, `cancelled`)
	`billCharges`	`GET org_bill_settings` (4 columns)
	`openOrders`	`GET orders` (+ `dining_tables`, `order_items`, `payments.amount`) for today, status not in (`paid`, `cancelled`), limit 100
	`refresh`	`GET orders?id&organization_id` (same embed)

Repository	Method	Operation
`LiveBillSettings`	`load` / `save`	`GET org_bill_settings`; `POST … on_conflict=organization_id` (ignore duplicates) / `PATCH …&updated_at=eq.<loaded>`
`LiveChannels`	`list` / `setAccepting`	`GET channel_integrations`; `PATCH channel_integrations {is_accepting, next_open_at: null}`
`LiveTeam`	`members` / `roles` / `setRole` / `setStatus`	`GET org_users` (+ `roles`); `GET roles`; `PATCH org_users {role_id}` or `{status}`
`LiveTables`	`list`	`GET dining_tables` (+ `dining_areas`)
`LiveStorefront`	`load` / `saveDraft` / `discardDraft`	`GET vinii_storefronts`, `GET vinii_storefront_versions` (limit 20); `POST vinii_storefronts on_conflict=organization_id`; `PATCH vinii_storefronts {draft:null}` (**both tables exist only in an unapplied draft migration**, see 15)

3. RPC (PostgREST `/rpc/<name>`)

RPC	Called from	Payload (`p`)	Defined?
`pos_place_order`	`LivePos.placeOrder` ← `SellCart.place` ← TicketView; `PosGuestGateway.place` ← `GuestSession.place`	see 02 §A7	**No**
`pos_record_payment`	`LivePos.recordPayment` ← `PaymentCard`	`order_id, method, amount, idempotency_key, reference, expected_updated_at`	**No**
`pos_set_order_status`	`LivePos.setStatus` (no caller)	`order_id, to, expected_updated_at, reason?`	**No**
`pos_adjust_bill`	`LivePos.adjustBill` ← `PosOrderScreen._adjust`	`order_id, expected_updated_at, waive_service_charge, waive_reason?`	**No**
`pos_issue_table_qr`	`LiveTables.issueCode` ← table codes screen	`table_id, replace?`	**No**
`vinii_publish_storefront`	`LiveStorefront.publish` ← Studio	`organization_id, definition, expected_version`	**Draft only, not applied** (Repo-1- `feat/experience-cms` 0021)

The call shapes are documented as a proposal in `docs/PLATFORM_CONTRACT_ALIGNMENT.md` §4.1, §4.4 and §4.5.

4. Storage

Call	Code
`storage.from('item-images').uploadBinary('<orgId>/<itemId>-<ms>.<ext>', bytes, contentType, upsert:false)`, then `getPublicUrl`	`live_menu.dart:213-222`. The app enforces a 5 MB limit before uploading

5. One Local partner API (`PARTNER_API_BASE_URL/api/partner`)

Headers: `Authorization: Bearer <Supabase access token>`, JSON, plus the caller headers. Writes also carry a new `idempotency-key` on each request.

Status mapping (`partner_api_inbox.dart:92-125`):

HTTP status	Shown as
401	auth
403	forbidden
404, 400, 413	rejected
409	conflict
426, 503	not ready

Method · path	Phone code	Server handler → database ([ledger] OL-ui)
`GET /me`	`outlet()`, which also refuses if the server's organization differs from the one chosen in the app (`:135–148`)	`requirePartnerAuth` reads `org_users`; outlet from `ol_merchant_channels`
`PATCH /outlet`	`setAccepting`	update `ol_merchant_channels`
`GET /orders?tab=new|preparing|ready|past`	`list` (polled)	`rpc ol_expire_unanswered(org)`, then `ol_orders ⋈ orders ⋈ order_items`
`GET /orders/<posOrderId>`	`detail`	`ol_orders` + `order_events`
`POST /orders/<id>/accept|reject|ready|out|delivered`	`accept`, `reject`, `markReady`, `markOut`, `markDelivered`	`rpc ol_partner_act(...)`, plus dispatch hooks on accept and reject
`POST /test-order`	`sendTestOrder`	`rpc ol_create_restaurant_order(...)`, then push

Partner endpoints this app does not call:

- `GET/PATCH /items`: One Local channel availability.
- `PUT /devices`: push registration.

6. Edge functions

None. `functions.invoke` does not appear anywhere in `lib/`.

7. Third-party endpoints

The Cashfree JS SDK (`https://sdk.cashfree.com/js/v3/cashfree.js`) is loaded only by the web build's checkout. That checkout is unreachable today, as no gateway supplies `payments` (04 §3).

08 — Realtime

Scope. `vinii_partner/` at `da0a823`. Evidence labels are as in `01_DATABASE.md`.

Database configuration ([ledger] Repo-1-)

Table	Added to `supabase_realtime`	Migration
`orders`	yes	0006, line 448
`kot_tickets`	yes	0006, line 449
`order_items`	yes	0010, line 397
`order_events`	yes	0010, line 398
`service_requests`	yes	0010, line 399
`table_sessions`	yes	0010, line 400

[unverified] Whether production's publication matches. The snapshot does not list publication members.

App usage: none

- `lib/` contains no `.channel(`, `.stream(`, `onPostgresChanges` or Realtime subscription. This was confirmed with a full search.
- Every "live" behaviour is **polling over HTTP**:

Screen	Mechanism	Interval	Evidence
Delivery inbox (new orders, ringing)	`Timer` → `GET /api/partner/orders?tab=new`	5 s while accepting, 30 s while not	`application/inbox_controller.dart:77–88`
Guest live tracking	`Future.delayed` loop → `LivePos.refresh`	5 s while the order is open	`application/guest_gateway.dart:82–94`
POS order screen, open orders, menu, tables	**no refresh**: read on open, or when the user pulls or taps	—	—

Side effects of polling

- Each inbox poll makes One Local's server run `ol_expire_unanswered(org)` first (`restaurant.ts:49`). So **the phone's poll is what times out unanswered orders and triggers auto-pause.** This matches the controller's own comment at `inbox_controller.dart:16–19`.
- Both pollers stop on sign-out or organization switch (`_reset`) and when the guest order closes.

Push notifications: none received. No push package is in `pubspec.yaml`, and the app never registers with `PUT /api/partner/devices`.

09 — Hardware

Scope. `vinii_partner/` at `da0a823`. Evidence labels are as in `01_DATABASE.md`.

Dependencies (`vinii_partner/pubspec.yaml`):

- `supabase_flutter`, `go_router`, `shared_preferences`, `intl`, `uuid`, `image_picker`, `audioplayers`, `qr_flutter`, `google_fonts`, `http`, `pdf`, `printing`, `web`.
- There is **no** Bluetooth, USB, serial, ESC/POS, NFC, barcode or scanner, payment-terminal or cash-drawer package.
- A search of `lib/` for `bluetooth`, `printer`, `esc_pos`, `serial`, `nfc`, `scanner`, `barcode` and `cash drawer` found no integration code.

Device	Status	Evidence
Thermal receipt printer	**Not integrated**	—
KOT printer	**Not integrated.** No KOT print or reprint	—
Barcode scanner	**Not integrated**	—
Cash drawer	**Not integrated**	—
Card or payment terminal	**Not integrated**	—
Bluetooth	**Not used**	—
Network printer	**Not integrated**	—
System print dialog (PDF)	**Used only for table QR codes.** `Printing.layoutPdf` prints an A4 sheet of table QR cards; a single card can be downloaded as a picture on web	`presentation/table_codes/code_exporter.dart:3,90`, `table_sheet_pdf.dart`
Speaker	Plays the new-order ring (`assets/sounds/new_order.wav`) with `audioplayers`	`application/ring_player.dart:16–60`
Vibration	`HapticFeedback.heavyImpact()` on a periodic `Timer` while ringing; Settings says vibration always stays on	`ring_player.dart:25–27`, `presentation/settings/settings_screen.dart:310`
Camera / photo library	`image_picker` for menu item photos, uploaded to `item-images`	`presentation/menu/item_editor_screen.dart:329`, `live_menu.dart:192–226`
Screen	Guest mode turns the staff phone into a guest ordering screen	02 §B

10 — Row security and permission enforcement

Scope. `vinii_partner/` at `da0a823`. Evidence labels are as in `01_DATABASE.md`. Policies were read in the migration files ([ledger]). The snapshot only gives policy *counts*, so production's policy text is [unverified].

1. Who the app is, to the database

- **Supabase client:** the **publishable (anon) key** plus the staff member's signed-in session, so the database role is `authenticated`.
- **No service-role key in the app.** `tool/run_local.sh` refuses any build setting outside a short allow-list: `APP_ENV`, `SUPABASE_URL`, `SUPABASE_PUBLISHABLE_KEY`, `PARTNER_API_BASE_URL`, `STOREFRONT_BASE_URL`, `MEDIA_BASE_URL`.
- **Partner API:** sent the same session's access token as a bearer token. One Local's server then acts with **its own service role** (`OL-ui/frontend/server/partner.ts`, `db()`).

2. "Your organization" in the database

- `current_org_id()` ([ledger] Repo-1- 0004) is security definer: `select organization_id from org_users where user_id = auth.uid() and status = 'active' limit 1`.
- There is **no `order by`**, so for a person with several active memberships the result is **one arbitrary organization**.
- Almost every member policy compares `organization_id = current_org_id()`. The app adds explicit `organization_id` filters (`live_backend.dart:35–37`). Those filters can narrow the result but never widen it.
- **Partner API uses a different rule:** the **earliest** active membership by `created_at` (`partner.ts:29–35`). The two choices can differ. The app detects that and refuses to show delivery orders (`partner_api_inbox.dart:135–148`).

3. Policies on the tables this app touches ([ledger])

Table	Select	Insert	Update	Delete	Role checked by the database?
`organizations`	platform admin; **member of the org (any active membership)** (0004)	platform admin	platform admin; `is_org_admin(id)`	—	update only: slug `org_admin` (**not `org_owner`**)
`org_users`	platform admin; `organization_id = current_org_id()`	`is_org_admin`	`is_org_admin`	`is_org_admin`	**yes**, writes: `org_admin` only
`roles`	everyone signed in (`using (true)`)	—	—	—	—
`org_modules`	`current_org_id()`	platform admin	platform admin	platform admin	—
`role_module_access`	`organization_id is null or = current_org_id()`	platform admin	platform admin	platform admin	—
`menu_categories`, `menu_items`, `dining_tables`	member	member	member	**member**	**no**: any member, including `kitchen` and `viewer`
`orders`	member	member	member	platform admin only	**no**
`order_items`	member (through parent order)	member	member	**member**	**no**
`kot_tickets`	member	member	member	**member**	**no**
`dining_areas`, `order_events`, `payments`, `org_bill_settings`, `channel_integrations`, `item_channel_status` (`apply_org_rls`, 0010)	member	member	member	**no policy, so denied** (delete is granted, but no policy allows it)	**no**
`org_counters`	member	no policy	no policy	—	writes only through `next_org_seq`
`vinii_storefronts`, `vinii_storefront_versions`	**not applied.** The draft `0021_vinii_storefronts.sql` defines member select, editor insert and update, versions written only by the publish function				draft checks roles (`vinii_can_edit_storefront`, `vinii_can_publish_storefront`)

"Member" means `organization_id = current_org_id()`. Platform admins have an `… for all` policy on every one of these tables.

Database-level consequences (as written in the ledger)

- **Any member** can call the REST API directly to:
  - insert, update or cancel an order;
  - change its totals or move its status backwards;
  - insert payments;
  - change menu prices;
  - change GST or service-charge rates;
  - switch channels on or off.
- Nothing in the database ties these to a role. The 0006 header says so: role checks are "enforced at the application layer for now".
- The CHECK lists limit status **values**, but nothing limits status **moves**. No trigger exists.

4. Functions

- `next_org_seq(org, kind)` is **security definer and executable by `authenticated`** (0006). It does **not** check that the caller belongs to `org`, so any signed-in user can advance any organization's order or KOT counter. [ledger]; whether this has been exploited is [unverified].
- `ol_create_restaurant_order`, `ol_partner_act` and `ol_expire_unanswered` are `revoke all … from public, anon, authenticated` and granted to `service_role` only (OL-ui 0023). The phone can reach them only through One Local's API.
- The `pos_*` functions the app calls do not exist on any branch, so their checks cannot be audited. `vinii_publish_storefront` exists only in the unapplied draft (Repo-1- `feat/experience-cms` 0021).

5. Storage: `item-images` ([ledger] Repo-1- 0006)

- Public read.
- Insert, update and delete are allowed for **any signed-in user, anywhere in the bucket**: the policy checks only `bucket_id = 'item-images'`.
- The app writes under `<orgId>/…` with `upsert: false`. That is a convention, not something the database enforces: one organization's member could overwrite or delete another organization's photos through the API.

6. Enforcement in the app: shows or hides buttons only

`lib/domain/staff.dart:136–175`, `capabilityRoles`. Screens call `session.can(Capability.x)`. Tabs also follow `org_modules` and `role_module_access` (`domain/app_definition.dart`).

Capability (app)	Roles in the app	Actual enforcement
`acceptOrders`	owner, admin, manager, location manager, operations manager, biller	**Server-enforced** by the partner API's `CAN.accept` and `CAN.reject` (`partner.ts:10–18`), with the same list
`markReady`	the above + `kitchen`	server `CAN.ready`: same
`dispatchOrders` (out, delivered)	the above, without `kitchen`	server `CAN.out` and `CAN.delivered`: same
`toggleOutlet`	owner, admin, manager, location manager, operations manager	server `CAN.outlet`: same
`sendTestOrder`	the above + `biller`	**Not enforced by the server.** `POST /test-order` checks only that the caller is signed-in staff; it has no `require()` (`app/api/partner/test-order+api.ts:5`)
`viewOrders`	most roles	server `view` → any active staff
`editMenu`, `toggleAvailability`	owner, admin, manager	**App only.** The database allows any member (§3)
`editBillSettings`	owner, admin	**App only.** The database allows any member
`toggleChannels`	owner, admin, manager	**App only.** The database allows any member

Capability (app)	Roles in the app	Actual enforcement
`manageTeam`	`org_admin`	**Database-enforced** (`is_org_admin`): same
`sell`, `takePayment`, `waiveServiceCharge`, `issueTableCodes`, `publishStorefront`	various	Meant for the missing RPCs; the database would have to check them. Today the calls fail before any check runs

7. Personal data on the phone

- **Customer phone and full address:** the partner API returns them only while an order is active (`accepted`, `packed`, `out`). Otherwise it returns the area only and the first name only (`restaurant.ts`, `shape()`).
- **Phone POS:** for takeaway and delivery it sends the customer's name and phone as free text. On a database built from the ledgers nothing is stored, because the RPC is missing.
- **Guest delivery:** the address is folded into the order notes (`guest_session.dart:425–431`).

11 — Entity ownership

Scope. `vinii_partner/` at `da0a823`. Evidence labels are as in `01_DATABASE.md`.

How to read the columns

- **Created / Updated / Read here** describe what *this app* does.
- **"via API"** means the phone asks One Local's partner API, and that server writes with its service role.
- **"via RPC (missing)"** means the app calls a database function that no migration defines (01 §6), so nothing happens today.

Entity	Table	Created here?	Updated here?	Read here?	Source of truth	Notes
Person (login)	`auth.users`	No	No (password reset email only)	Yes (session)	Supabase Auth	One login across apps
Membership	`org_users`	No (invites are not possible from the app)	**Yes**: `role_id`, `status`	Yes	Database	Database allows only `org_admin` to write (`is_org_admin`)
Role	`roles`	No	No	Yes	Database (Repo-1-seeds)	—
Organization	`organizations`	No	No	Yes: id, name, slug, type, status, theme, settings	Database / Master	`settings.app.min_app_version` and `settings.app.tabs` drive the update gate and tab order
Module switches	`org_modules`, `role_module_access`	No	No	Yes	Database / Master	—
Location / branch	`locations`	No	No	**No**	Database	Not used anywhere in the app
Menu category	`menu_categories`	**Yes**	**Yes**: name, sort order, delete	Yes	Database	Reordering is one update per row, not atomic
Menu item	`menu_items`	**Yes**	**Yes**: every editable field, `is_available`	Yes	Database	Also edited by the Repo-1- web console and possibly the Windows POS (not audited)
Item photo	storage `item-images`	**Yes** (upload)	No (new file per upload)	Yes (public URL)	Storage	Old files are never deleted
Item variant	`menu_item_variants`	No	No	**No**	Database	Invisible to the phone
Item availability per channel	`item_channel_status`	No	No	No	Database	Written by One Local's `/items`, which this app does not call
Dining table	`dining_tables`	No	No	Yes	Database	—
Table QR code	`dining_tables.qr_token`	via RPC (missing) `pos_issue_table_qr`	via RPC (missing)	Yes	Database	Globally unique column
Dining area	`dining_areas`	No	No	Yes	Database	—
Phone POS order	`orders`	via RPC (missing) `pos_place_order`	via RPC (missing) `pos_adjust_bill`, `pos_record_payment`; `pos_set_order_status` has no caller	Yes (open orders, order screen, table occupancy)	Database	Today the phone cannot create one
One Local order (kitchen side)	`orders` (`channel online`, `placed_by integration`)	No (One Local; test order via API)	via API (`ol_partner_act`)	via API; **also directly**, in the Open-orders read (12 §1)	Database	—
One Local order (customer side)	`ol_orders`	No	via API	via API	Database (One Local)	The phone shows `ol_orders.status`, not `orders.status`
Order line	`order_items`	via RPC (missing) / One Local	via API (status, `kot_ticket_id`)	Yes (embedded)	Database	—

Entity	Table	Created here?	Updated here?	Read here?	Source of truth	Notes
KOT	`kot_tickets`	via API (accept)	via API (ready, out)	**No**	Database	—
Order timeline	`order_events`	via API (One Local functions)	No	via API (`GET /orders/<id>`)	Database	—
Payment	`payments`	via RPC (missing) `pos_record_payment`; via API (delivered → cash row)	No	Yes (`amount`, embedded)	Database	—
Bill settings	`org_bill_settings`	**Yes** (first save, upsert)	**Yes** (version-checked update)	Yes	Database	The phone's bill estimate uses them; the server is meant to price
Sales channel switch	`channel_integrations`	No	**Yes**: `is_accepting`, `next_open_at = null`	Yes	Database	The Repo-1- control panel writes the same columns
One Local outlet switch	`ol_merchant_channels`	No	via API (`PATCH /outlet`); auto-pause by server	via API (`GET /me`)	Database (One Local)	—
Delivery dispatch job	`delivery_jobs`	No	indirectly (accept or reject hooks on One Local's server)	No	Database (One Local 0029)	[unverified] whether applied in production
Storefront (draft and published versions)	`vinii_storefronts`, `vinii_storefront_versions`	**Yes** (upsert), if the table existed	**Yes** (draft); publish via RPC (missing)	Yes, if the table existed	**The phone's SharedPreferences, while the tables are missing**	Tables exist only in the unapplied draft `0021_vinii_storefronts.sql` (Repo-1- branch `feat/experience-cms`)
Customer	**none**	No	No	No	**None.** Free-text `customer_name` and `customer_phone` on the order	No customer table or id. One Local has `ol_orders.customer_hash` and `user_id` on its side
Device	**none**	No	No	No	—	No device id is generated or sent. `x-platform` and `x-runtime` headers only. The app never registers in `ol_partner_devices`
Idempotency key	SharedPreferences `pending_key:*`	**Yes** (UUID v4)	cleared on a definite answer	Yes	The phone	Never stored in the database: no column exists
Cart	memory (`SellCart`)	Yes	Yes	Yes	The phone, until sent	Lost on restart, sign-out or organization switch

12 — Duplicates and conflicts

Scope. `vinii_partner/` at `da0a823`. Evidence labels are as in `01_DATABASE.md`. Each item is something observed in code or migrations, not a risk inferred in general.

1. Orders

1. **One Local orders appear in two places on the phone.**
   - The Orders tab reads them through the partner API, keyed by `ol_orders.status`.
   - The Sell tab's "Open orders" sheet reads `orders` directly: today, status not in (`paid`, `cancelled`) (`live_pos.dart:180–201`). That includes One Local's kitchen-side rows (`channel = 'online'`).
   - **The same order therefore shows in both lists.** The Sell side shows it as a POS order: `channel 'online'` falls through to `ServiceType.delivery` (`live_pos.dart:321–325`).
   - A **delivered** One Local order ends with `orders.status = 'delivered'`, not `paid` (OL-ui 0023, delivered branch). So it **stays in "Open orders" for the rest of the day**, even though it is paid in full.
2. **Two order-number streams share one counter.** One Local's function prints `OL–` + 5 digits from `next_org_seq(org, 'order')`. Any other writer of `orders` draws from the same per-organization counter. Numbers cannot collide (unique `(organization_id, order_no)`), but each channel's displayed numbers will have gaps.
3. **No idempotency on `orders`.** The phone sends keys (06), but the `orders` table has no column for them, and `external_ref` is **not unique** (Repo-1- 0010). One Local's test order sends no key at all. Pressing "Send test order" twice creates two orders, by design.
4. **Different "order created" logic.**
   - `ol_create_restaurant_order` hard-codes GST at 5% and adds the delivery fee into `orders.total`.
   - The phone's estimate reads `org_bill_settings` (SGST + CGST, service charge, rounding) (`sell_cart.dart`, `domain/billing.dart`).
   - The phone POS's pricing is meant to be the missing `pos_place_order`.
   - So there are already **two pricing rules** for rows in the same `orders` table.

2. Customers

No customer entity exists on this app's side. The same customer is stored as free text on every order (`orders.customer_name` and `customer_phone`). On One Local's side the same person is `ol_orders.customer_hash` or `user_id`. Nothing links the two.

3. Products

- **Two availability switches** with overlapping meaning:
  - `menu_items.is_available`, which applies everywhere and is written by this app;
  - `item_channel_status` for `one_local`, written by One Local's `/items` and **not visible in this app**.
  - An item can be "available" on the phone and "off" for One Local, or the reverse. Nothing on the phone shows the second switch.
- **Several editors of `menu_items`:** this app, the Repo-1- web console, and [unverified] the Windows POS. None of them uses a version check, so the last write wins (06).
- **Variants:** `menu_item_variants` defines prices that the phone never reads. The phone always uses `menu_items.price`, which the table comment calls the "starts at" price when variants exist.

4. Inventory

No inventory exists (05), so there are no duplicates.

5. Payments

- **No idempotency in the database.** `payments` has no key column and `reference` is not unique. Duplicate protection depends entirely on the missing `pos_record_payment` (phone) or on `ol_partner_act`'s state check (One Local delivery).
- **Two ways to mark "paid":**

    - One Local's delivered path sets `orders.payment_method = 'cash'`, `paid_at`, status `delivered`, and inserts a payment row.
    - The phone treats an order as paid when `status = 'paid'` or the payments add up to the total (`domain/pos_order.dart:228`).
    - Neither path sets the other's marker.
- **The cash-on-delivery row's `amount` equals `orders.total`**, which includes One Local's delivery fee. Restaurant takings counted from `payments` therefore include money that belongs to the delivery fee.

6. KOTs

- **Two status fields for the same kitchen work:** `kot_tickets.status` and `order_items.status`, plus `orders.status` and `ol_orders.status`. One Local's function updates all of them together. Nothing in the database keeps them consistent for other writers (no trigger).
- **`order_events.kind` repeats state** already held in the status columns. It has no check list, so any string is accepted.

7. Identity and organization choice

Two different rules for "which organization is this person in":

- the database's `current_org_id()`: `limit 1`, no order;
- One Local's partner API: earliest active membership.

For a person with two memberships they can disagree. The app then refuses the delivery inbox (`partner_api_inbox.dart:135–148`).

8. Configuration

- **Module availability is read two ways.**
    - The `org_modules` table comment says "Absence of a row means the module is not available to the org at all" (Repo-1-0007).
    - The app treats a missing row for `dashboard`, `orders`, `pos` or `menu` as **on** (`domain/app_definition.dart:77–81`).
- **The storefront draft can live in two places:** the `vinii_storefronts.draft` column (if it existed) and SharedPreferences `storefront_draft:<orgId>`. `load` prefers the database and falls back to the phone (`live_storefront.dart:61–67`).

13 — Unknown or unverified

Scope. `vinii_partner/` at `da0a823`. This audit read source code and migration files. It did **not** connect to any database, run the app, or call any API. Everything below is either unknown or was not checked.

Production database

1. **Whether production matches the ledgers.** The only production evidence is the column-list snapshot of 2026-09-22 (`OL-ui/docs/CENTRAL_DB_MAP.md`). It matches the ledger columns for `orders`, `order_items`, `kot_tickets`, `order_events`, `payments`, `org_bill_settings`, `org_users`, `roles`, `role_module_access` and `org_modules`. It gives policy **counts** only, not policy text.
2. **Production functions after 2026-09-22.** The snapshot reports 29 functions without naming them. No migration on any branch defines `pos_place_order`, `pos_record_payment`, `pos_set_order_status`, `pos_adjust_bill` or `pos_issue_table_qr`. `vinii_publish_storefront` exists only in the unapplied draft (item 3). Whether someone created any of them by hand in production is unknown.
3. **Whether `vinii_storefronts` and `vinii_storefront_versions` exist in production.** Only the unapplied draft `0021_vinii_storefronts.sql` (Repo-1- branch `feat/experience-cms`) creates them.
4. **Which One Local migrations (OL-ui 0021–0029) are applied in production.**
   - Earlier documents say 0021–0024 are live and 0028 was pending.
   - 0025–0027 and 0029 (dispatch, `delivery_jobs`) are unknown.
   - The snapshot's `ol_orders` column list (2026-09-22) predates 0022's added columns. Whether production now has `organization_id`, `restaurant_deadline`, `prep_minutes`, `gst` and `cancel_reason` is unknown.
5. **The members of the realtime publication in production.**
6. **The `organizations.type` enum in production.**
   - The ledger defines `org_type` as (`franchise`, `investor`).
   - The app treats `type` as a business kind and falls back to `'restaurant'` (`live_workspace.dart:53`).
   - Whether production's enum was widened is unknown.
7. **The `item-images` storage policies in production** (ledger: any signed-in user, whole bucket).

Other applications that write the same tables (not audited)

8. **The Windows POS** (`proviyaa-pos`): how and whether it writes `orders`, `order_items`, `kot_tickets` and `payments` in the shared database (its own decision records mention PowerSync and an outbox). It is the most likely writer of KOT `preparing` and of dine-in status moves. **Not inspected in this audit.**
9. **The Repo-1- web console** (`apps/web/src/lib/pos.ts`, for example `setKotStatus` and `settleOrder`): its writes to the same tables were noted in earlier alignment work but not re-read here.
10. **One Local's dispatch code** (`OL-ui/frontend/server/dispatch.ts`): only the function names called on accept and reject were confirmed, not their database effects.
11. **One Local's partner API at runtime:**
    - The route-to-function mapping was read in source (`restaurant.ts`, `partner.ts`, `app/api/partner/*`).
    - The deployed version, and whether it matches source, is unknown.
    - `PATCH /outlet` (`restaurant.ts:105-115`) and `notifyNewOrder` were read only in outline.

Behaviour that depends on missing server functions

12. Everything after "Send to kitchen", "Save & hold", "Charge", "Remove service charge", "Issue code" and "Publish" on the phone. The app's expected response shapes are documented (`live_pos.dart:287-348`; `docs/PLATFORM_CONTRACT_ALIGNMENT.md` §4), but what the database would do cannot be verified.
13. Whether a server would honour `coupon_code`, `waive_service_charge`, `scheduled_for` and `fire_kot` (sent in `pos_place_order`). The `orders` table has no columns for a coupon, a waiver or `scheduled_for`.

Client details not exercised

14. **Where `supabase_flutter` persists the session on each platform.** Not checked.

15. **Cashfree checkout.** Its own comment says it was "not yet run against Cashfree's sandbox" (`cashfree_checkout_web.dart:19–22`). It is unreachable today in any case.
16. **Tests.** `vinii_partner/test/` was not run for this audit. It uses a fake backend (`test/support/fake_backend.dart`) and does not touch a database.